GUIA PRÁTICO • 29 DE OUTUBRO DE 2026

Guia Oficial do Uso de Skills

Como instalar e usar as **11 skills do repositório**: qual chamar, o que informar, o que ela entrega e qual é o próximo passo.



Neste guia	Página
1. Visão Geral e Arquitetura de Skills	2
2. Inventário de Skills • objetivo e uso das 11 skills	3-4
3. Como Utilizar as Skills em um Projeto Real	5-9
A. Preparando o Projeto	5
B. Adicionando uma Feature	6-7
C. Deploy e Operação	8-9

Como ler os fluxos

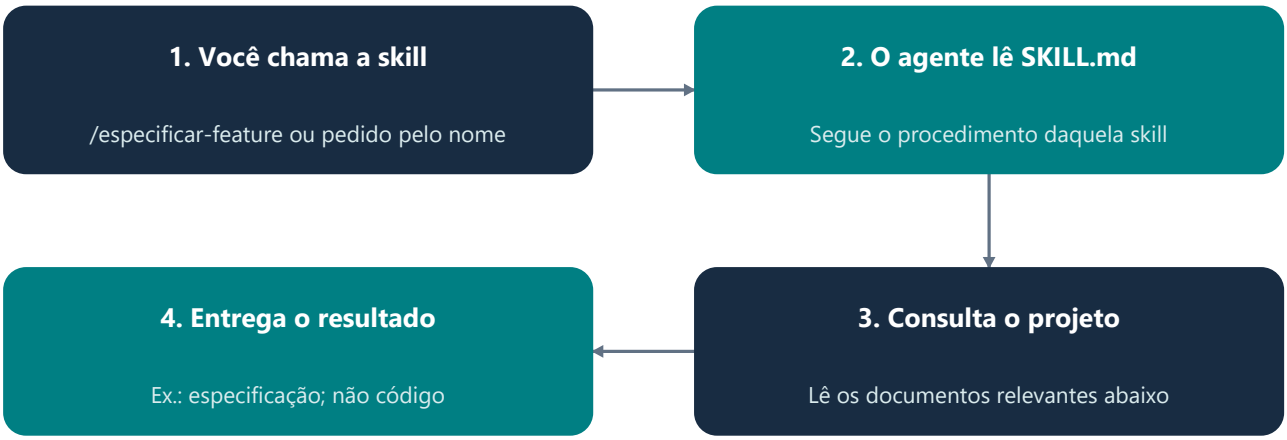
Blocos escuros: etapas principais. **Contorno tracejado:** etapa opcional. **Bloco âmbar:** decisão ou condição de avanço. As setas indicam a ordem, não autorização automática para executar.

Fonte do conteúdo e dos fluxos: README.md e SKILL.md do repositório local, incluindo mudanças não commitadas. O PDF anexado foi usado exclusivamente como referência de design: diagramação, hierarquia visual e apresentação. Os exemplos de chamados ilustram pedidos às skills.

1. VISÃO GERAL E ARQUITETURA DE SKILLS

Como uma skill funciona no seu projeto

Cada skill é uma pasta com **SKILL.md**: nome, descrição de quando usar e instruções de execução. O catálogo é independente de produto e tecnologia. As regras e decisões específicas são lidas nos arquivos do projeto em que você chama a skill.



Arquivo do projeto	Como as skills o utilizam
AGENTS.md / CLAUDE.md	Instruções comuns; CLAUDE.md importa @AGENTS.md.
HARNESS.md	Regras particulares do projeto, com precedência sobre o modelo.
PRD.md	Escopo, requisitos, histórias e regras de negócio.
TDD.md	Stack, arquitetura, interfaces e configuração.
DESIGN.md	Estilo visual quando a tarefa envolve interface.
BDD.md + bdd/	Índice e cenários com IDs para especificação e geração de testes.

Duas formas de chamar

NO CHAT DO AGENTE

/especificar-feature

Ou: “Use a skill especificar-feature para detalhar o encerramento de chamados.”

Use o comando com barra se o agente oferecer esse suporte. Caso contrário, peça pelo nome. O README declara suporte a Claude Code, Cursor, Copilot, Devin Desktop e agentes compatíveis com SKILL.md.

2. INVENTÁRIO DE SKILLS

Preparar, especificar e construir

Invoque **/nome-da-skill** onde o agente aceitar esse formato, ou escreva “use a skill nome-da-skill”. As fichas abaixo resumem o contrato; o SKILL.md continua sendo a referência operacional.

01 • iniciar-projeto

Objetivo: Instalar instruções e criar o contexto inicial.
Na prática: “Use iniciar-projeto na raiz e preserve documentos existentes.” Cria arquivos ausentes e verifica CLI do host, Context7 e Octocode.
Limite: Não define produto nem implementa; diferenças no modelo exigem escolha de manter, mesclar ou substituir.

02 • especificar-projeto

Objetivo: Definir PRD, TDD e o primeiro recorte do produto.
Na prática: “Especifique um sistema de chamados; vamos definir o PRD antes da stack.” Escolha stack, revisão cruzada, design com frontend e BDD opcional.
Limite: Não implementa. Se o projeto já está especificado, só refaz mediante pedido.

03 • implementar-sdd

Objetivo: Orquestrar especificação, plano e implementação.
Na prática: “Implemente o encerramento de chamados usando implementar-sdd.” Aprove o plano e depois escolha testes BDD, revisão e commit.
Limite: Não inicia deploy. Na implementação, não altera testes; sua geração é oferecida depois.

04 • especificar-feature

Objetivo: Transformar uma intenção em história, regras e critérios.
Na prática: “Especifique o encerramento de chamados, incluindo erros e cenários.” A spec sai no chat; salvar arquivo e gravar BDD requer autorização.
Limite: Não implementa. O plano é produzido no modo de planejamento nativo do agente.

05 • criar-testes

Objetivo: Gerar os testes ausentes para cenários automatizáveis.
Na prática: “Crie testes para os IDs sem cobertura em bdd/us-02.md.” Aprove o mapeamento cenário → arquivo; cada teste usa o mesmo ID.
Limite: Não muda produção, cenários ou testes já associados a IDs. @manual fica fora da automação.

06 • revisar-codigo

Objetivo: Detectar problemas e sugerir melhorias por criticidade.
Na prática: “Revise o diff e os testes da feature contra o TDD.” Examina duplicação, SOLID, arquitetura, convenções e resultados do BDD.
Limite: Não edita nem refatora. Aplicar uma sugestão exige outro pedido.

2. INVENTÁRIO DE SKILLS

Versionar, entregar e evoluir

07 • commitar

Objetivo: Organizar commits, ajustar versão e enviar a branch.
Na prática: "Organize as mudanças por assunto e mostre o plano antes de enviar." Executa checagens, agrupa arquivos e registra a versão em commit próprio.
Limite: Não cria tag ou release. No fluxo unificado, não oferece documentação ou PR ao final.

08 • gerar-guardrails

Objetivo: Preparar CI de PR e proteção da branch no GitHub.
Na prática: "Reaproveite o CI e exija o check verde." Aprove workflows e regras; ative a proteção depois do primeiro CI bem-sucedido.
Limite: Uma vez, somente com testes configurados. Não commita nem faz push. Fora do GitHub, entrega um plano.

09 • atualizar-docs

Objetivo: Sincronizar PRD, TDD e README com o código real.
Na prática: "Atualize a documentação e, ao final, commite." Escolha no início entre commit, release ou nenhum; revise ambiguidades apontadas.
Limite: Não altera código ou BDD. Para release, commita primeiro a documentação pendente.

10 • publicar-release

Objetivo: Criar e enviar a tag da versão já declarada.
Na prática: "Publique a versão do manifesto e acompanhe o pipeline." Confira branch, notas, checagens e tag antes de confirmar.
Limite: Exige árvore limpa. Não commita nem eleva versão; sem pipeline, a tag não publica artefatos automaticamente.

11 • gerar-skill

Objetivo: Criar um procedimento abstrato e reutilizável.
Na prática: "Extraia uma skill de revisão de contratos de API." Defina gatilho e limites, aprove o rascunho e depois decida sobre o PR para o catálogo.
Limite: Não implementa a tarefa descrita nem copia escolhas locais para todos os projetos.

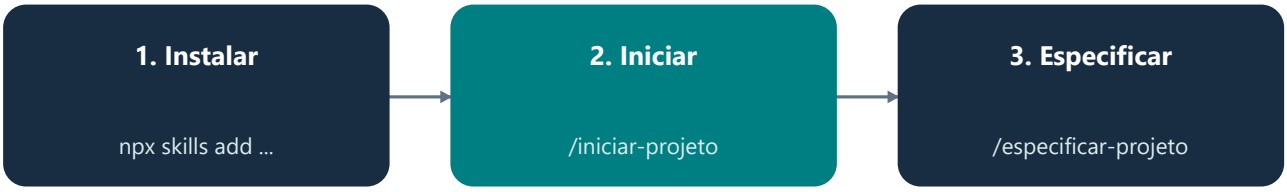
QUAL SKILL ESCOLHER?

Feature inteira → implementar-sdd
Só requisitos → especificar-feature
Só qualidade → criar-testes ou revisar-codigo
Fechar entrega → atualizar-docs + commitar ou publicar-release

3. COMO UTILIZAR AS SKILLS EM UM PROJETO REAL

A. Preparando o Projeto

EXECUTA SOMENTE UMA VEZ NO INÍCIO



1. Instalando Skills

Na raiz do projeto que consumirá as skills, use o comando do README:

INSTALAÇÃO

```
npx skills add paulocesaaars/skills -a claude-code -a cursor -a github-copilot --copy -y
```

Versione **.claude/skills/**, **.agents/skills/** e **skills-lock.json**. Para instalação global, acrescente **-g**; para uma skill, **-s nome-da-skill**. Atualize com **npx skills update -y** e revise o diff. A orquestradora precisa ter disponíveis as skills que chama.

2. Iniciando Projeto

Use **/iniciar-projeto**. A skill prepara AGENTS.md, CLAUDE.md, HARNESS.md, PRD.md, TDD.md, DESIGN.md, BDD.md e bdd/ quando faltarem. Preserva conteúdo existente e verifica ferramentas. Pastas de aplicação, como src/ e tests/, serão definidas na arquitetura; não são criadas por essa etapa.

3. Especificando Projeto

Use **/especificar-projeto**. Primeiro delimite se o repositório é o projeto inteiro ou parte de um maior. Depois siga a sequência:



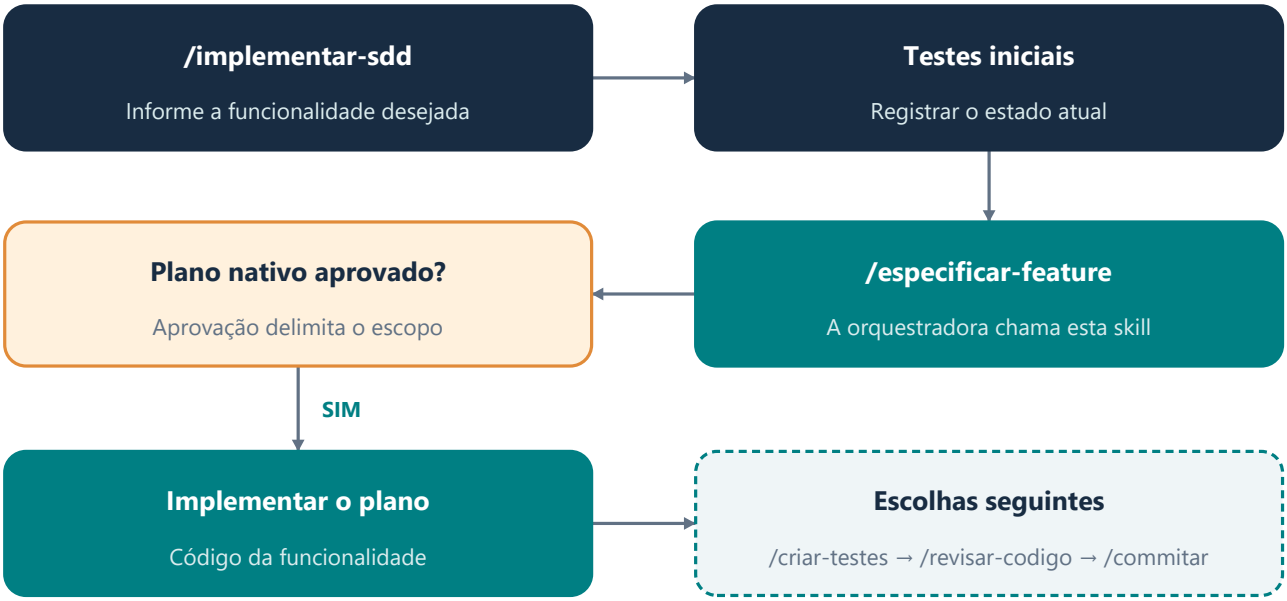
Primeira especificação: informe objetivo, público, o que entra e o que fica fora da entrega inicial. Exemplo de pedido: "Use especificar-projeto para registrar, listar e encerrar chamados; deixe anexos fora." O repositório não define uma etapa chamada MVC; se você quiser a arquitetura Model-View-Controller, explicite essa escolha no TDD.

Saída: documentos coerentes, decisões críticas resolvidas e critérios aprovados. Arquivos apenas criados e vazios não significam projeto especificado.

3 • B. ADICIONANDO UMA FEATURE

1. Especificando Feature

Escolha o caminho: implementar-sdd coordena o fluxo completo; no caminho manual, execute as skills individualmente. Planejamento e implementação continuam necessários entre a especificação e as opções de validação.



Sem plano aprovado, a implementação não começa.

O que informar à skill

PEDIDO E EXEMPLO DE RESULTADO

Use especificar-feature para o atendente encerrar um chamado aberto. Não inclua reabertura.

Exemplo de cenário a validar: @US-02.1
Dado um chamado aberto e um atendente autorizado
Quando ele solicita o encerramento
Então o chamado fica com status encerrado

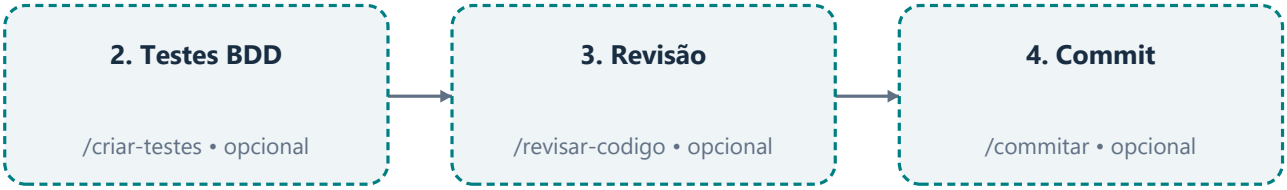
Defina também erros, permissões e exclusões. Use IDs estáveis em bdd/ e mantenha BDD.md como índice. A gravação dos cenários depende de aprovação; a especificação só vira arquivo depois de você autorizar o destino.

Caminho manual: após especificar-feature, ative o planejamento nativo, aprove o plano e peça sua execução. Não há uma skill separada de implementação. **Caminho unificado:** implementar-sdd faz essa coordenação e oferece as opções da página seguinte.

Atenção de adoção: o modelo de AGENTS.md pede testes junto da implementação, enquanto implementar-sdd os separa como opção posterior. Alinhe essa política no HARNESS.md antes de usar o fluxo; não resolva a divergência silenciosamente.

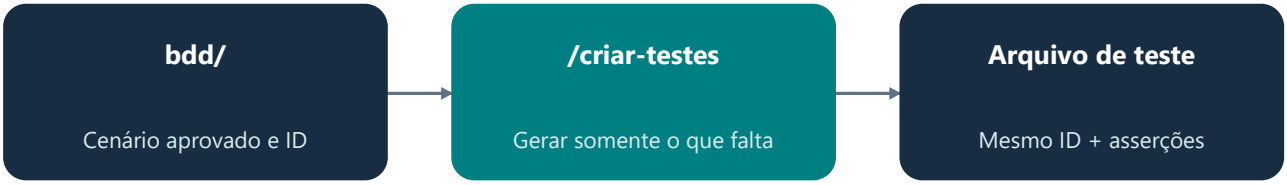
3 • B. ADICIONANDO UMA FEATURE

Validar e fechar a funcionalidade



2. Gerar Teste do BDD (Opcional)

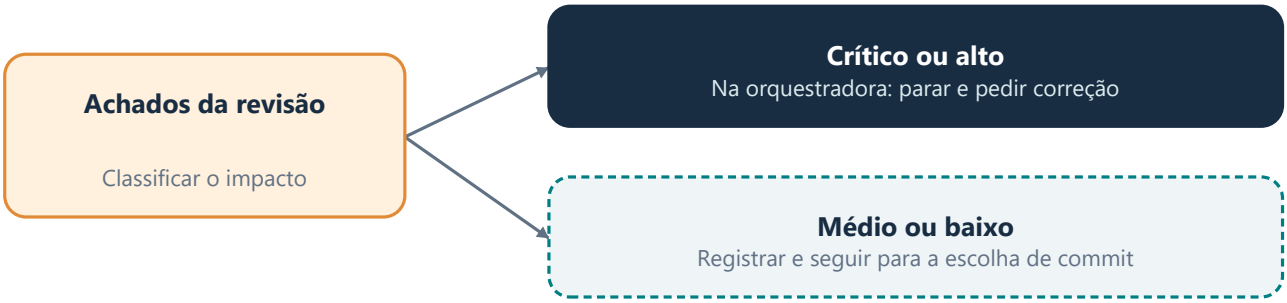
Use **/criar-testes** para cenários sem teste. A skill descobre runner e marcação, mostra o plano e cria um teste por cenário com o mesmo ID. Não reescreve os já associados; @manual vira checklist.



Se o teste revelar falha do produto, preserve cenário e asserção. criar-testes não corrige produção; na orquestradora, você decide se volta à implementação para corrigir a lacuna.

3. Revisar Código (Opcional)

Use **/revisar-codigo** no diff e nos testes novos. Avalie duplicação, SOLID, arquitetura, cobertura dos resultados e trade-offs. A skill entrega problema, impacto e sugestão; **não aplica refatorações**.



4. Comitar (Opcional)

Use **/commitar**. Revise grupos por assunto, arquivos excluídos, branch e versão antes de aprovar. A skill testa, cria commits coesos, ajusta a versão quando aplicável e envia a branch; **não cria tag**. Exemplo: **feat(chamados): adicionar encerramento**.

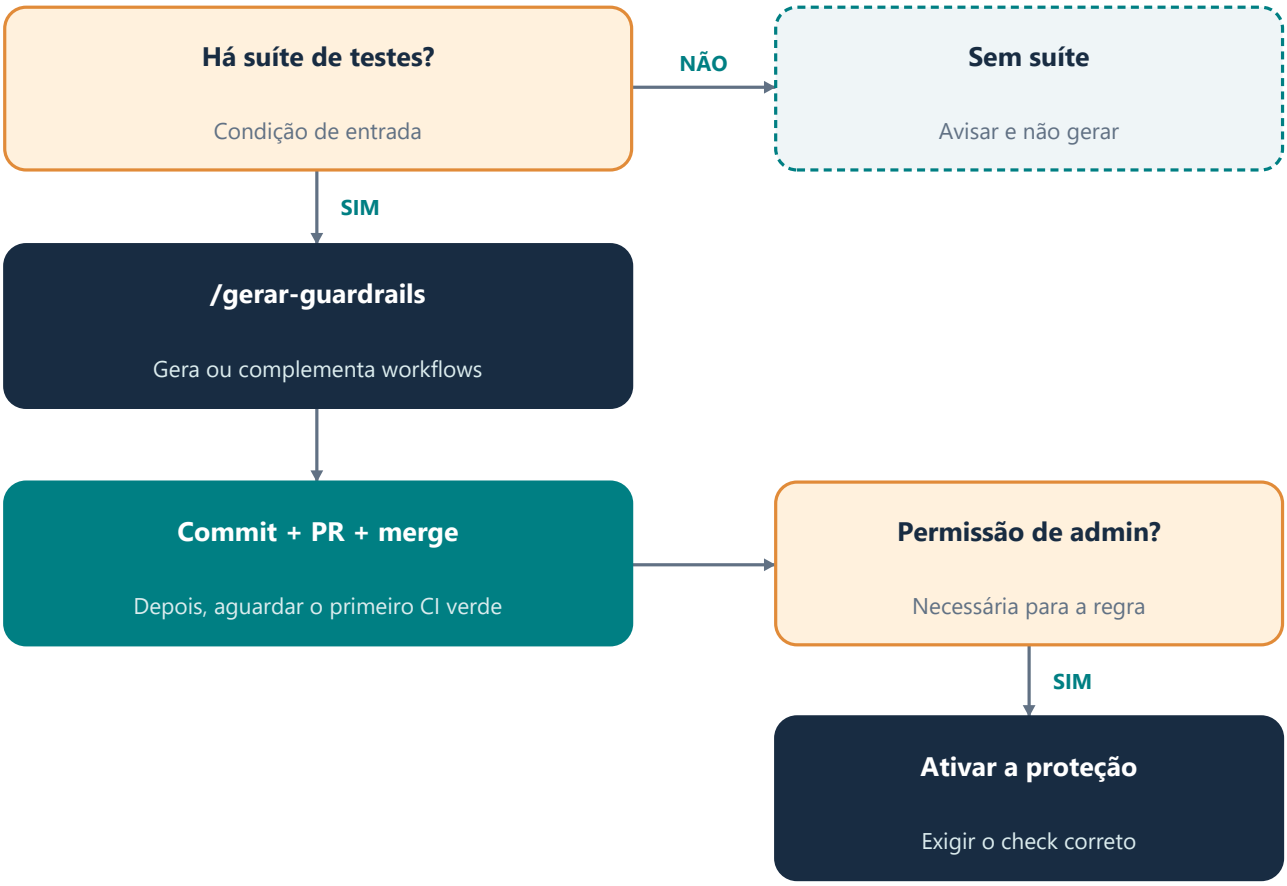
Bloqueios: testes ou checagens falhando impedem commit e release. Sem suíte, as skills avisam e seguem, sem declarar funcionamento validado. Pular uma etapa opcional não equivale a tê-la concluído.

Fim do ciclo B: implementar-sdd não oferece documentação ou PR e não começa deploy. Inicie a fase C com um pedido separado.

3 • C. DEPLOY E OPERAÇÃO

1. Gerar Guardrails (Opcional)

Somente uma vez e somente com testes configurados. Peça: “Use gerar-guardrails para exigir os testes do projeto no CI do GitHub”. A skill preserva workflows existentes e mostra o plano antes de alterar arquivos ou regras remotas.



SE A REGRA NÃO PUDER SER ATIVADA

Sem permissão administrativa: entregue o pedido ao administrador e registre a proteção como pendente. Não declare a branch protegida apenas porque o YAML existe.

Por que a ordem importa

Exigir um check antes de sua primeira execução pode deixar PRs aguardando um resultado inexistente. Por isso, o workflow deve chegar à principal e executar antes da proteção. gerar-guardrails não faz commit nem push; essa passagem usa o fluxo normal de versionamento.

Se CI e proteção já exigem testes verdes, não gere novamente. Em outros hosts, a skill entrega o plano, sem executar mudanças remotas. A disponibilidade das regras também depende dos recursos da conta.

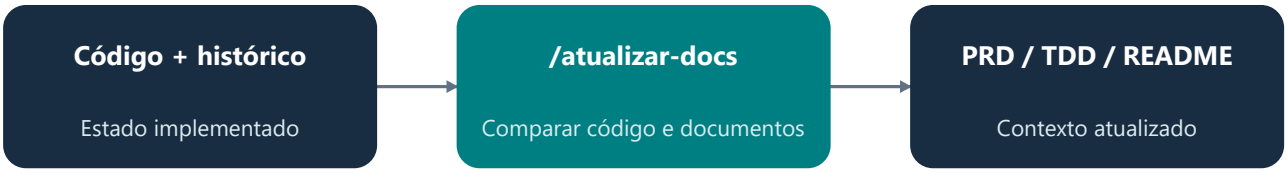
Verificação: YAML válido, comandos locais executados, nome exato do check e regra ativa. O contrato atual pode aceitar “nenhum teste coletado”: CI verde, nesse caso, não comprova testes executados.

3 • C. DEPLOY E OPERAÇÃO

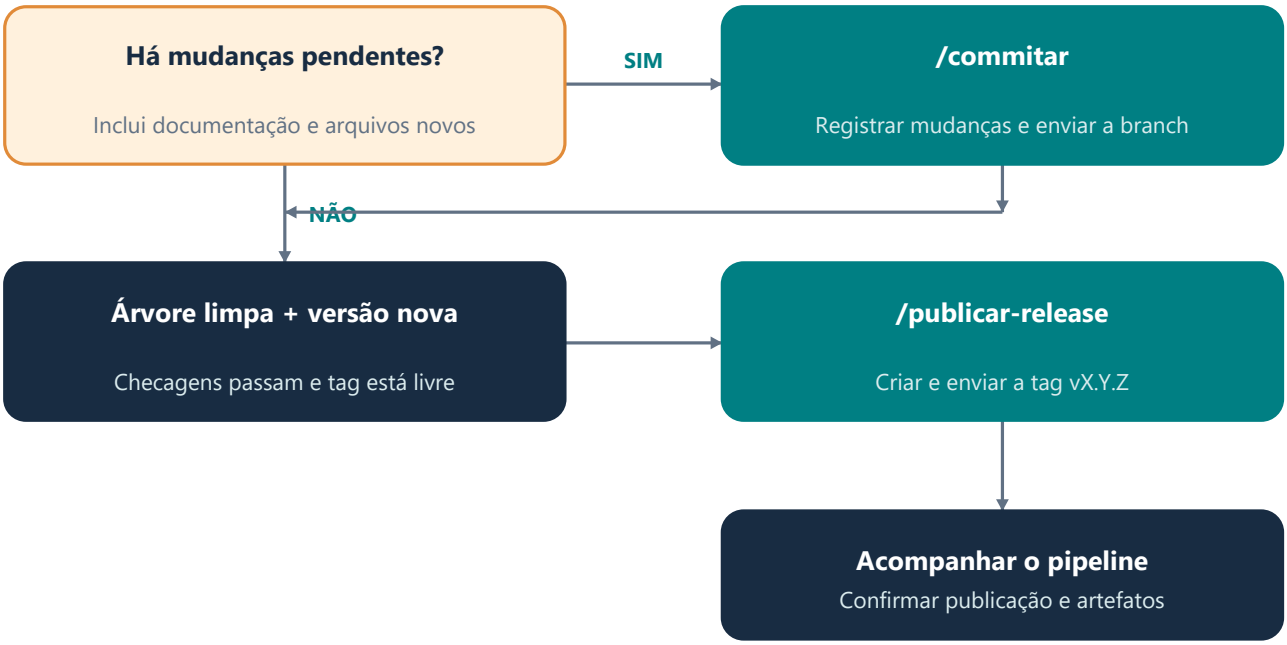
Documentar e publicar

2. Atualizar Docs

Use **/atualizar-docs**. Escolha no início se deseja commit, release ou nenhum. A skill compara código e histórico, preserva edições locais e sincroniza **PRD** (produto), **TDD** (técnico) e **README** (uso). BDD não é editado; lacunas viram pendências.



3. Comitar ou Publicar Release



O fluxo acima vale quando você escolhe publicar. Se escolher somente commit, ele termina após /commitar; se escolher nenhum, termina após atualizar os documentos. Para release, atualizar-docs commita apenas seus arquivos alterados, sem elevar a versão novamente, e depois chama publicar-release. Outras pendências ainda impedem a release.

Confira o resultado: publicar-release lê a versão do manifesto e não a altera. Sem pipeline de release, o envio da tag não publica artefatos automaticamente. Com pipeline, verifique sua conclusão e o link da release antes de considerar a publicação encerrada.

Próximo ciclo: volte a B para a próxima feature, preservando contexto e guardrails. Não repita a preparação a cada mudança.

Referências: README e catálogo paulocesaaars/skills; contratos locais de skills/*/SKILL.md e modelo de AGENTS.md. Exemplos de chamados são ilustrativos.